

Embedded Systems Programming

Sessional - III

January 29, 2025

0.1 Introduction to I2C

I2C stands for *Inter-Integrated Circuit*. It is a serial communication protocol designed by Philips Semiconductor in 1982. I2C is widely used for short-distance communication between integrated circuits (ICs) on the same board. It is also known as the Two-Wire Interface (TWI).

Two-Wire Communication: I2C uses only two bi-directional lines, Serial Data (SDA) and Serial Clock (SCL), for communication. SDA is used for data transfer, while SCL carries the clock signal.

Master-Slave Configuration: I2C operates in master-slave mode, where the master initiates communication and the slave responds. Multiple masters and slaves can be connected on the same bus.

Half-Duplex Communication: I2C supports bi-directional communication but not simultaneously, making it a half-duplex protocol. Synchronous Communication: Data transfer is synchronized with the clock signal, ensuring reliable communication.

Addressing: Each device on the I2C bus has a unique address. The master sends the address of the target slave device before initiating communication.

Error Handling: I2C uses ACK/NACK bits for error detection and handling. After each data byte, the receiver sends an acknowledgment (ACK) or not acknowledgment (NACK) bit.

The I2C bus is one of the most commonly used microcontroller communication protocols for communicating with external devices such as sensors and actuators. The bus is a single master, multiple slave bus and it can operate in multiple modes.

Because the I2C bus is based on just two wires, there should be a way to address an individual slave device on the same bus. For this reason, the protocol defines that each slave device provides a unique slave address for the given bus. This address is usually 7-bits wide. When the bus is free, both lines are HIGH. All communication on the bus is initiated and completed by the master which initially sends a START bit, and completes a transaction by sending a STOP bit. This alerts all the slaves that some data is coming on the bus and all the slaves listen on the bus. After the start bit, 7 bits of the unique slave address are sent. Each slave device on the bus has its own address and this ensures that only the addressed slave communicates on the bus at any time to avoid any collisions. The last sent bit is a read/write bit

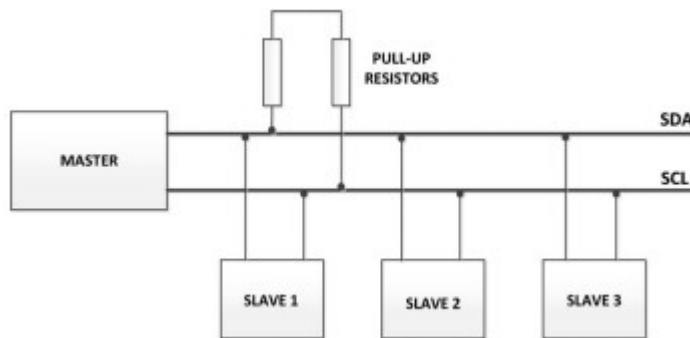


Figure 1: A schematic representation of the I2C bus; SDA: data line, SCL: clock line

such that if this bit is 0, it means that the master wishes to write to the bus (e.g. to a register of a slave), if this bit is a 1, it means that the master wishes to read from the bus (e.g. from the register of a slave). The data is sent on the bus with the MSB bit first. An acknowledgment (ACK) bit takes place after every byte and this bit allows the receiver to signal to the transmitter that the byte was received successfully and as a result, another byte may be sent. ACK bit is sent at the 9th clock pulse.

0.2 I2C pins on the Raspberry Pi Pico

Raspberry Pi Pico has two I2C pins, named I2C0 and I2C1. As shown in the figure, the pins are duplicated and shared with other pins. For example, GP0 (pin 1) is the I2C0 SDA pin and GP1 (pin 1) is the I2C0 SCL pin. Also, GP16 (pin 21) is the I2C0 SDA pin and GP17 (pin 22) is the I2C SCL pin.

0.3 Available API function call

The I2C API commonly allows for the following function calls, most of which are self explanatory:

1. `clear()`
2. `putstr()`
3. `backlight_on()`

4. `backlight_off()`
5. `show_cursor()`
6. `hide_cursor()`
7. `blink_cursor_on()`
8. `blink_cursor_off()`
9. `display_on()`
10. `display_off()`
11. `move_to(column, row)` – left hand corner is (0, 0)
12. `putchar()`
13. `custom_char()`

0.4 Assignments

1. Put a string on a I2C LCD panel
2. Display a running counter on the I2C pannel, where the numbers updates every 1 second.
3. Generate a random number between 1 to 5. Display the same on a LCD screen and light up a corresponding LED.
4. Print the multiplication table of a number line by line on a LCD panel.
5. Print a line of scrolling text on LCD panel.
6. Simulate the bouncing ball game. The output of your program should how many jumps were required for the ball to land on the target location. The game starts of with a target having a given co-rdinate i.e. (x,y). The ball is placed randomly at a point on the screen and then jumps about randomly. You win when the ball lands on the target square within N moves, else you lose the game.